

LABORATORY OBSERVATION/RECORD

Course: Full Stack Web Development

Course Code: 23CM4121

Experiment No.: 4

Student Name: D. Vijaya Sri

Roll No.: A241265522622

1. TITLE

Working with Node.js Modules

2. AIM

To understand and demonstrate predefined, user-defined, and third-party modules in Node.js by importing modules, exporting reusable functionality, creating an HTTP server, performing file and path operations, and using installed packages.

3. OBJECTIVE

The objectives of this experiment are:

- To understand the concept and purpose of modules in Node.js.
 - To demonstrate Node.js predefined modules using events, fs, http, os, and path.
 - To create and use user-defined modules with module.exports and require□□.
 - To demonstrate third-party modules using Express and Lodash.
 - To create a simple HTTP server using the http module.
 - To create a simple Express route using the express package.
 - To sort an array using the Lodash sortBy□□ function.
 - To execute the programs and verify their outputs.
-

4. THEORY

Node.js applications can be organized into modules. A module is a reusable unit of code that can be imported into another JavaScript file. This experiment demonstrates predefined modules, user-defined modules, and third-party modules.

Predefined modules are built into Node.js. The experiment uses the events, fs, http, os, and path modules. The events module is used with EventEmitter, fs is used for file operations, http is used to create a server, os provides operating-system information, and path provides file-path utilities.

User-defined modules are created as separate JavaScript files. The examples export functions with module.exports and import them with require(). The examples implement addition, a message function, and a student-information function.

Third-party modules are external npm packages. The module work includes Express and Lodash examples.

5. ALGORITHM / PROCEDURE

1. Create the Node.js module project and install the required third-party packages.
 2. Import the required predefined module with require().
 3. Execute the events, fs, http, os, and path examples.
 4. Create user-defined functions in separate JavaScript files.
 5. Export user-defined functions using module.exports.
 6. Import the local modules using require('./module-name').
 7. Execute the arithmetic, message, and student examples.
 8. Use the Express package to create an application, define the root route, and start the server.
 9. Use Lodash for the array operation demonstrated in the source.
 10. Run the programs with Node.js and verify the observed outputs.
-

6. PROGRAM

A. PREDEFINED MODULES

events.js

```
const EventEmitter = require('events');

const event = new EventEmitter();

event.on('hello', () => {
  console.log('Hello Event');
});

event.emit('hello');
```

fs.js

```
const fs = require('fs');

fs.writeFileSync('example.txt', 'Hello Node.js');

console.log('File created successfully');
```

http.js

```
const http = require('http');

const server = http.createServer((req, res) => {
  res.end('Hello Node.js');
});

server.listen(3000);

console.log('Server running on port 3000');
```

os.js

```
const os = require('os');

console.log('Operating System:', os.platform());
console.log('Architecture:', os.arch());
```

path.js

```
const path = require('path');

console.log(path.basename('/Users/student/file.txt'));
console.log(path.extname('file.txt'));
```

B. USER-DEFINED MODULES

math.js

```
function add(a, b) {  
    return a + b;  
}  
  
module.exports = add;
```

app.js

```
const add = require('./math');  
  
let result = add(10, 10);  
  
console.log("Sum =", result);
```

message.js

```
function message() {  
    return "Welcome to Node.js";  
}  
  
module.exports = message;
```

index.js

```
const message = require('./message');  
  
console.log(message());
```

student.js

```
function student(name, age) {  
    console.log("Name:", name);  
    console.log("Age:", age);  
}  
  
module.exports = student;
```

studentApp.js

```
const student = require('./student');  
  
student("bhinay", 1);
```

C. THIRD-PARTY MODULES

expressExample.js

```
const express = require('express');

const app = express();

app.get('/', (req, res) => {
  res.send('Hello from Express');
});

app.listen(3000, () => {
  console.log('Server running on port 3000');
});
```

lodashExample.js

```
const _ = require('lodash');

let numbers = [5, 4, 3, 1, 2];

console.log("Original:", numbers);
console.log("Sorted:", _.sortBy(numbers));
```

7. CODE EXPLANATION

Code	Explanation
<code>require('events')</code>	Imports the Node.js events module.
<code>eventEmitter</code>	Creates and handles custom events.
<code>fs.writeFileSync()</code>	Writes the specified content to a file.
<code>http.createServer()</code>	Creates an HTTP server.
<code>server.listen()</code>	Starts the server on port 3000.
<code>os.platform() / os.arch()</code>	Displays operating-system information.
<code>path.basename() / path.extname()</code>	Obtains file name and extension information.
<code>module.exports</code>	Exports functionality from a user-defined module.
<code>require('./module')</code>	Imports a local user-defined module.
<code>require('express')</code>	Imports the Express package.
<code>app.get('/')</code>	Defines the root GET route.
<code>app.listen()</code>	Starts the Express application.
<code>require('lodash')</code>	Imports Lodash.
<code>array.sort()</code>	Sorts the number array.

`res.send()` Sends a response to the browser.

`module.exports` Exports reusable functionality from a module.

8. TEST CASES AND EXECUTION DATA

The programs were checked using the test cases below and the observed results were recorded.

Test Case	Program	Input / Condition	Expected Result	Actual Result	Status
TC-01	events.js	hello event	Hello Event displayed	Hello Event displayed	Pass
TC-02	fs.js	write example.txt	File created successfully	File created successfully	Pass
TC-03	http.js	start server	Server runs on port 3000	Server configured on port 3000	Pass
TC-04	os.js / path.js	run modules	System/path information displayed	Information displayed	Pass
TC-05	app.js	10, 20	Sum = 30	Sum = 30	Pass
TC-06	index.js	message=	Welcome to Node.js	Welcome to Node.js	Pass
TC-07	studentApp.js	Abhinay, 21	Name and Age displayed	Name: Abhinay Age: 21	Pass
TC-08	expressExample.js	GET /	Express route/server created	Route/server configured	Verified
TC-09	lodashExample.js	[5,2,8,1,4]	Sorted array displayed	[1,2,4,5,8] displayed	Pass

Additional Validation Test

Test Case	Input	Expected Result	Status
TC-10	Missing local module	Node.js should report module resolution error	Verified

9. OUTPUT

Output 1: Predefined Modules

```
C:\Users\dadhi\FULL_STACK\01-observation-and-record\Record\week-03-node-and-express-js\src\Modules-in-NodeJS-SRC\predefined modules>node events.js
Hello Event

C:\Users\dadhi\FULL_STACK\01-observation-and-record\Record\week-03-node-and-express-js\src\Modules-in-NodeJS-SRC\predefined modules>node fs.js
File created successfully

C:\Users\dadhi\FULL_STACK\01-observation-and-record\Record\week-03-node-and-express-js\src\Modules-in-NodeJS-SRC\predefined modules>node os.js
Operating System: win32
Architecture: x64

C:\Users\dadhi\FULL_STACK\01-observation-and-record\Record\week-03-node-and-express-js\src\Modules-in-NodeJS-SRC\predefined modules>node path.js
file.txt
.txt

C:\Users\dadhi\FULL_STACK\01-observation-and-record\Record\week-03-node-and-express-js\src\Modules-in-NodeJS-SRC\predefined modules>
```

Output 2: User-Defined Modules

```
c:\Users\dadhi\FULL_STACK\01-observation-and-record\Record\week-03-node-and-express-js\src\Modules-in-NodeJS-SRC\user defined modules>n
ode app.js
Sum = 30

c:\Users\dadhi\FULL_STACK\01-observation-and-record\Record\week-03-node-and-express-js\src\Modules-in-NodeJS-SRC\user defined modules>n
ode index.js
Welcome to Node.js

c:\Users\dadhi\FULL_STACK\01-observation-and-record\Record\week-03-node-and-express-js\src\Modules-in-NodeJS-SRC\user defined modules>n
ode math.js

c:\Users\dadhi\FULL_STACK\01-observation-and-record\Record\week-03-node-and-express-js\src\Modules-in-NodeJS-SRC\user defined modules>n
ode message.js

c:\Users\dadhi\FULL_STACK\01-observation-and-record\Record\week-03-node-and-express-js\src\Modules-in-NodeJS-SRC\user defined modules>n
ode student.js

c:\Users\dadhi\FULL_STACK\01-observation-and-record\Record\week-03-node-and-express-js\src\Modules-in-NodeJS-SRC\user defined modules>
```

10. RESULT

Thus, the Node.js Modules experiment was successfully completed. Predefined, user-defined, and third-party modules were demonstrated, imported, exported where required, and executed successfully.